APACHE KAFKA SERIES

Kafka Streams for Data Processing

A Complete Hands-On, Chapter-by-Chapter Deep Dive

High-Level DSL | Exactly-Once Semantics | No Cluster to Manage | Stateful & Stateless Processing | Joins, Aggregations, Testing

Table of Contents

Chapter 1 — Kafka Streams — First Look
What is Kafka Streams · Prerequisites · Meet the Instructor · Your First App · Kafka Streams vs. Spark/NiFi/Flink

Chapter 2 — End-to-End Kafka Streams Application — WordCount
Core Concepts · Environment Setup · Topology Build · Packaging · Scaling

Chapter 3 — KStreams and KTables — Simple Operations (Stateless)
map/mapValues · filter · flatMap · branch · selectKey · Repartitioning · Duality

Chapter 4 — KStreams and KTables — Advanced Operations (Stateful)
groupBy · count/aggregate/reduce · peek · transform · Writing to External Systems

Chapter 5 — Exactly-Once Semantics — Theory
What is EOS · Idempotent Producer & Transactions · Kafka CLI Hands-On · Spring Boot Hands-On

Chapter 6 — Exactly-Once Semantics — Practice Exercise: BankBalance
Producer + Streams App Build, Guidance, Solutions & Verification

Chapter 7 — Joins — KStream to GlobalKTable & KTable to GlobalKTable
Join Types · Co-partitioning · UserEnrich · Foreign-Key Enrichment · Update Behavior

Chapter 8 — Testing Your Kafka Streams Application
TopologyTestDriver · Unit Testing WordCount · Self-Guided Exercises

Chapter 9 — KStream vs. KTable — The Big Idea in Plain English
Stateless vs. Stateful Ops · The Golden Rule · Joins · Cheat Sheet

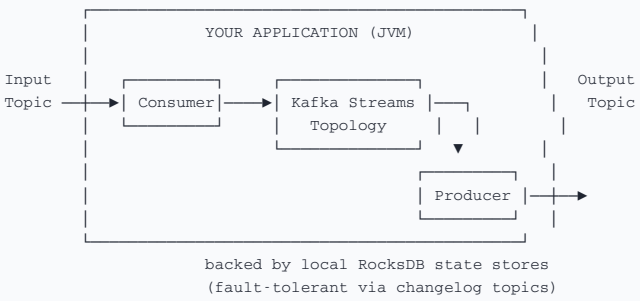
CHAPTER 1

Kafka Streams — First Look

1.1 What is Kafka Streams?

Kafka Streams is a client library — **not a separate cluster or infrastructure component** — for building applications and microservices that process and transform data stored in Apache Kafka. You write a normal Java (or Scala) application, add the `kafka-streams` dependency, and your application reads from one or more input topics, applies transformations, and writes results to output topics (or to a database, an API, a cache — anywhere you like).

The defining idea is: *the stream processing logic lives inside your own application's JVM*. There is no separate "Streams cluster" to install, configure, or babysit. Kafka itself (the brokers you already run) provides the durability, fault tolerance, and scalability — your application simply becomes another well-behaved Kafka client, using consumers and producers under the hood.



Why Kafka Streams exists

Before Kafka Streams, developers who wanted stream processing on top of Kafka typically reached for a separate framework — Spark Streaming, Apache Flink, Apache Storm, or Apache NiFi — each of which required its own cluster, its own deployment model, and its own operational learning curve. Kafka Streams removes that friction: if you can package and run a Java application (as a JAR, in a container, behind a load balancer, however you like), you can run a Kafka Streams application. Scaling out is as simple as starting more instances of the same JAR with the same `application.id`.

Key properties

Property	What it means in practice
No separate cluster	You deploy your app the same way you deploy any Java service — Docker, Kubernetes, bare JVM, whatever your organization already uses.
Elastic scaling	Parallelism is driven by the number of partitions in the input topic(s). Start more app instances (or threads) and Kafka rebalances the partitions across them automatically.
Fault tolerance	Local state is continuously backed up to internal Kafka "changelog" topics, so a crashed instance's state can be rebuilt elsewhere.
Exactly-once processing	Built directly on Kafka's transactional producer/consumer APIs (Kafka 0.11+) — one configuration flag away.
Two APIs	The high-level **DSL** (map, filter, join, aggregate...) and the low-level **Processor API** for full control.

1.2 Course Objective / Prerequisites / Target Students

Course objective. By the end of this material you should be able to design, write, test, package, and run production-style Kafka Streams applications: stateless transformations, stateful aggregations, joins between streams and tables, and applications that guarantee exactly-once processing.

Prerequisites

- Comfort with core Apache Kafka concepts: topics, partitions, producers, consumers, consumer groups, offsets.
- Working Java knowledge (Java 8+ syntax, including lambda expressions — we use them constantly).
- Basic familiarity with Maven (or Gradle) for dependency management and packaging.
- An IDE — IntelliJ IDEA (Community Edition is sufficient) is used throughout this guide.
- A local Kafka broker you can start/stop freely (Confluent Platform, Apache Kafka binaries, or Docker Compose all work).

Who this is for

> **TARGET AUDIENCE**
> Developers and data engineers who already run Kafka producers/consumers and now need to build real transformation, enrichment, or aggregation logic without introducing a second big-data framework into their stack.

1.3 About Your Instructor

This course is built in the style of Stephane Maarek's widely used Apache Kafka series — a certified Kafka instructor and consultant known for pragmatic, hands-on Kafka training that emphasizes running real code over slideware. The philosophy carried through this guide is the same: every concept is paired with a runnable example.

1.4 Running Your First Kafka Streams Application: WordCount

The traditional "Hello World" of stream processing is **WordCount**: read lines of text from an input topic, split them into words, and continuously output a running count of each word. It's deceptively simple, but it touches almost every core concept — stateless transformation (splitting text), stateful aggregation (counting), and continuous output to a compacted topic.

🧪 Hands-On Lab 1.1 — Run WordCount End to End

Goal: download Kafka, start a single-node cluster, and run the built-in WordCount demo before writing any code yourself.

```
# 1. Download and extract Apache Kafka
wget https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.7.0.tgz
tar -xzf kafka_2.13-3.7.0.tgz
cd kafka_2.13-3.7.0

# 2. Start Zookeeper (or use KRaft mode — see callout below)
bin/zookeeper-server-start.sh config/zookeeper.properties

# 3. In a new terminal, start the broker
bin/kafka-server-start.sh config/server.properties

# 4. Create input and output topics
bin/kafka-topics.sh --create --topic streams-plaintext-input \
  --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1

bin/kafka-topics.sh --create --topic streams-wordcount-output \
  --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1 \
  --config cleanup.policy=compact

# 5. Run the demo WordCount application shipped with Kafka
bin/kafka-run-class.sh org.apache.kafka.streams.examples.wordcount.WordCountDemo

# 6. Produce some text into the input topic
bin/kafka-console-producer.sh --topic streams-plaintext-input --bootstrap-server localhost:9092

# 7. Consume the running word counts (note: keys are printed too)
bin/kafka-console-consumer.sh --topic streams-wordcount-output \
  --from-beginning --bootstrap-server localhost:9092 \
  --property print.key=true \
  --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
  --property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

Try typing `kafka streams kafka streams kafka` into the producer and watch the consumer output evolve: `kafka 1`, `streams 1`, `kafka 2`, `streams 2`, `kafka 3` — the count updates on every new record because the output topic represents a changelog of the running total, not a batch result.

ZOOKEEPER VS. KRAFT

Older Kafka distributions require Zookeeper to be started before the broker. Kafka 3.3+ also supports KRaft mode (no Zookeeper). Use whichever your Kafka version and course track expects — the Streams application code is unaffected either way.

1.5 Kafka Streams vs. Other Stream-Processing Libraries

It's worth understanding where Kafka Streams fits before committing to it for a project.

Framework	Deployment model	Best fit	Trade-off vs. Kafka Streams
Kafka Streams	Embedded library inside your own JVM app; no separate cluster	Kafka-to-Kafka transformations, microservices that react to events	Kafka-only: input/output must be Kafka topics (or state stores) — not a general-purpose multi-source engine
Spark Streaming (Structured Streaming)	Dedicated Spark cluster (YARN/K8s/standalone)	Large batch + streaming hybrid workloads, heavy ML integration	Micro-batch latency (hundreds of ms+), separate cluster to operate
Apache Flink	Dedicated Flink cluster (JobManager/TaskManagers)	True low-latency, complex event-time processing at very large scale	Steeper operational overhead; another distributed system to run
Apache NiFi	Standalone NiFi cluster with a flow-based UI	Data routing/ETL between many heterogeneous systems, minimal coding	Not aimed at complex stateful stream computation; more of an integration/routing tool

RULE OF THUMB

If your data both starts and ends in Kafka, and your team is comfortable writing Java/Scala, Kafka Streams is usually the lowest-friction choice — you avoid standing up an entirely separate cluster just to transform events that are already flowing through Kafka.

Chapter 1 Takeaways

- Kafka Streams is a **library**, not infrastructure — you deploy it as a normal application.
- It reads Kafka topics, transforms data via the DSL or Processor API, and writes back to Kafka (or elsewhere).
- WordCount is the canonical first example: stateless split + stateful count.
- Choose Kafka Streams when both source and sink are Kafka and you want to avoid operating a second cluster.

CHAPTER 2

End-to-End Kafka Streams Application — WordCount

2.1 Section Objective

In this chapter you build the WordCount application **yourself**, from an empty Maven project to a runnable, debuggable, packageable JAR — instead of just running the demo bundled with Kafka.

2.2 Kafka Streams Core Concepts

Concept	Definition
Topology	A directed acyclic graph of processing nodes (sources, processors, sinks) describing how data flows through your application.
KStream	An unbounded, append-only stream of records; every record is treated as an independent new event (an "insert").
KTable	A changelog / continuously-updated table; a new record with an existing key *overwrites* the previous value for that key (an "upsert").
State Store	A local (usually RocksDB-backed) key-value store used for stateful operations like counting or joining; backed by a Kafka changelog topic for fault tolerance.
StreamsBuilder	The DSL entry point used to declare a topology (`builder.stream(...)` , `builder.table(...)`).
Serde	Serializer/Deserializer pair — Kafka Streams needs to know how to (de)serialize both keys and values at every step.

2.3 Environment and IDE Setup: Java 8, Maven, IntelliJ IDEA

1. Install a JDK (Java 8 minimum — the DSL leans heavily on lambdas and functional interfaces introduced in Java 8).
2. Install Maven, or rely on IntelliJ's bundled Maven wrapper.
3. Install IntelliJ IDEA Community Edition and enable the Maven plugin (enabled by default).
4. Confirm your local Kafka broker is running (see Chapter 1, Lab 1.1) and reachable at `localhost:9092`.

2.4 Starter Project Setup (pom.xml)

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example.kafka</groupId>
    <artifactId>kafka-streams-wordcount</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <kafka.version>3.7.0</kafka.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.kafka</groupId>
            <artifactId>kafka-streams</artifactId>
            <version>${kafka.version}</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.9</version>
        </dependency>
    </dependencies>
</project>
```

2.5 Kafka Streams Application Properties

Every Kafka Streams application configures itself through a `Properties` object, analogous to producer/consumer configs but with a few Streams-specific keys.

```
Properties config = new Properties();
config.put(StreamsConfig.APPLICATION_ID_CONFIG, "wordcount-application");
config.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
config.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass());
config.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.String().getClass());
```

Property	Purpose

`APPLICATION_ID_CONFIG`	Doubles as the consumer group ID *and* as a prefix for internal topics (repartition & changelog topics). Must be unique per logical application.
`BOOTSTRAP_SERVERS_CONFIG`	The Kafka broker(s) to connect to.
`DEFAULT_KEY/VALUE_SERDE_CLASS_CONFIG`	Default serializer/deserializer applied whenever a step doesn't specify its own.

2.6 Java 8 Lambda Functions — Quick Overview

The Kafka Streams DSL is designed around functional interfaces, so lambda syntax keeps code compact:

```
// Traditional anonymous class
stream.filter(new Predicate<String, String>() {
    public boolean test(String key, String value) {
        return value.length() > 5;
    }
});

// Equivalent lambda — this is what we use throughout the course
stream.filter((key, value) -> value.length() > 5);
```

2.7 WordCount Application Topology

This is the heart of the chapter — the full DSL pipeline, step by step:

```
StreamsBuilder builder = new StreamsBuilder();

KStream<String, String> textLines = builder.stream("streams-plaintext-input");

KTable<String, Long> wordCounts = textLines
    // 1. lowercase everything
    .mapValues(value -> value.toLowerCase())
    // 2. split each line into a stream of words (one input record -> many output records)
    .flatMapValues(value -> Arrays.asList(value.split("\\W+")))
    // 3. discard the (meaningless) original key, re-key by the word itself
    .selectKey((ignoredKey, word) -> word)
    // 4. group by the new key so we can aggregate per word
    .groupByKey()
    // 5. count occurrences per key -> produces a KTable<String, Long>
    .count(Materialized.as("Counts"));

// stream the running counts back out to Kafka
wordCounts.toStream().to("streams-wordcount-output", Produced.with(Serdes.String(), Serdes.Long()));

Topology topology = builder.build();
KafkaStreams streams = new KafkaStreams(topology, config);
streams.start();
```



2.8 Printing the Kafka Streams Topology

Before starting the application it's good practice to print the topology description — it's the fastest way to sanity-check what you actually built versus what you intended:

```
System.out.println(topology.describe());
```

```
Topologies:
   Sub-topology: 0
    Source: KSTREAM-SOURCE-0000000000 (topics: [streams-plaintext-input])
      --> KSTREAM-MAPVALUES-0000000001
    Processor: KSTREAM-MAPVALUES-0000000001 (stores: [])
      --> KSTREAM-FLATMAPVALUES-0000000002
      <-- KSTREAM-SOURCE-0000000000
    Processor: KSTREAM-FLATMAPVALUES-0000000002 (stores: [])
      --> KSTREAM-KEY-SELECT-0000000003
      <-- KSTREAM-MAPVALUES-0000000001
    Processor: KSTREAM-KEY-SELECT-0000000003 (stores: [])
      --> KSTREAM-FILTER-0000000004
      <-- KSTREAM-FLATMAPVALUES-0000000002
    Processor: KSTREAM-FILTER-0000000004 (stores: [])
      --> KSTREAM-SINK-0000000005
      <-- KSTREAM-KEY-SELECT-0000000003
    Sink: KSTREAM-SINK-0000000005 (topic: wordcount-application-Counts-repartition)
      <-- KSTREAM-FILTER-0000000004

  Sub-topology: 1
    Source: KSTREAM-SOURCE-0000000006 (topics: [wordcount-application-Counts-repartition])
      --> KSTREAM-AGGREGATE-0000000007
    Processor: KSTREAM-AGGREGATE-0000000007 (stores: [Counts])
      --> KTABLE-TOSTREAM-0000000008
      <-- KSTREAM-SOURCE-0000000006
    Processor: KTABLE-TOSTREAM-0000000008 (stores: [])
      --> KSTREAM-SINK-0000000009
      <-- KSTREAM-AGGREGATE-0000000007
    Sink: KSTREAM-SINK-0000000009 (topic: streams-wordcount-output)
      <-- KTABLE-TOSTREAM-0000000008
```

READING THE OUTPUT

Notice the topology is split into **two sub-topologies** connected by an internal repartition topic. Because `selectKey` changed the key, Kafka Streams must re-partition the data by the new key before it can safely group and aggregate — otherwise records for the same word could land on different partitions/tasks and be counted incorrectly.

2.9 Kafka Streams Graceful Shutdown

A Streams application should close its resources (flush state stores, commit offsets) cleanly on shutdown rather than being killed abruptly:

```
Runtime.getRuntime().addShutdownHook(new Thread(streams::close));
```

WHY THIS MATTERS

Without a shutdown hook, a `Ctrl+C` or container stop signal can leave local RocksDB files in an inconsistent state, forcing a full state-store rebuild from the changelog topic on next startup — slower recovery, and in the worst case, temporarily duplicated processing.

2.10 Running the Application from IntelliJ IDEA

1. Right-click your `main` class (e.g. `WordCountApp`) → *Run*.
2. Confirm the console shows `State transition from CREATED to REBALANCING` then `REBALANCING to RUNNING` — this confirms the app has joined its consumer group and is actively processing.
3. Produce test data with `kafka-console-producer.sh` and confirm counts arrive on the output topic via `kafka-console-consumer.sh`.

2.11 Debugging the Application from IntelliJ IDEA

Set breakpoints directly inside your lambda bodies (e.g. inside the `mapValues` or `flatMapValues` lambda) and run in **Debug** mode instead of Run. Because each lambda executes once per record, IntelliJ will stop execution every time a record passes through that stage — invaluable for confirming your split/lowercase logic behaves as expected on messy real-world input.

2.12 Internal Topics for Our Kafka Streams Application

Kafka Streams automatically creates and manages two categories of internal topics, both prefixed with your `application.id`:

Internal topic type	Naming pattern	Purpose
Repartition topic	`<application.id>-<name>-repartition`	Re-keys/re-partitions data whenever the key changes (e.g. after `selectKey`) before a downstream grouping/join.
Changelog topic	`<application.id>-<store-name>-changelog`	A durable, compacted backup of a state store's contents, used to rebuild state after a crash or on scale-out.

You can inspect these with `kafka-topics.sh --list` — you'll see them alongside your explicitly created topics.

2.13 Packaging the Application as a Fat JAR & Running the Fat JAR

To run outside the IDE you need a single "fat" (uber) JAR containing your code plus all dependencies. The Maven Shade plugin is the standard tool:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.5.1</version>
      <executions>
        <execution>
          <phase>package</phase>
          <goals><goal>shade</goal></goals>
          <configuration>
            <transformers>
              <transformer implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                <mainClass>com.example.kafka.WordCountApp</mainClass>
              </transformer>
            </transformers>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

```
# build
mvn clean package

# run the resulting jar directly, no IDE required
java -jar target/kafka-streams-wordcount-1.0-SNAPSHOT.jar
```

2.14 Scaling Our Application

Because parallelism is tied to partitions, scaling out is mechanical: run additional instances of the *same* fat JAR (same `application.id`) on the same or different machines. Kafka Streams uses the consumer group protocol to automatically rebalance input-topic partitions — and the corresponding state — across all running instances.

> **PRACTICAL LIMIT**
> You can run more application instances than partitions, but the extras will sit idle as standby/hot spares — true throughput parallelism tops out at one active task per input partition. Plan your partition count with future scaling headroom in mind.

2.15 Section Wrap-Up

Chapter 2 Takeaways

- A Streams app is configured via `Properties`, built via `StreamsBuilder`, and started/stopped via `KafkaStreams`.
- `topology.describe()` is your best debugging friend before you ever produce a record.
- Key-changing operations (like `selectKey`) force an internal repartition topic — this is normal and required for correctness.
- Always register a shutdown hook for graceful `streams.close()`.
- Scaling = running more copies of the same fat JAR with the same `application.id`.

CHAPTER 3

KStreams and KTables — Simple Operations (Stateless)

3.1 Section Objectives

Cover the stateless DSL operators — transformations that process each record independently, with no memory of previous records — before moving on to stateful operations in Chapter 4.

3.2 KStream & KTable

	KStream	KTable
Semantics	Record stream — every event is independent ("insert" / fact)	Changelog — a new value for a key replaces the old one ("upsert" / latest state)
Analogy	A bank transaction log	A bank account balance
Null value	Ignored — just another record	Interpreted as a **tombstone** (delete that key)
Backed by	The topic itself	The topic *plus* a local state store

3.3 Stateless vs. Stateful Operations

Stateless operations (this chapter) transform each record in isolation — no state store is needed, and no data from earlier records influences the result. Examples: `map`, `filter`, `flatMap`, `branch`, `selectKey`.

Stateful operations (Chapter 4) need to remember information across records — counting, aggregating, reducing, joining — and therefore require a local state store, which in turn needs a changelog topic for fault tolerance.

3.4 mapValues / map

```
// mapValues: transform the value only, key untouched, 1-to-1
KStream<String, String> upperCased = stream.mapValues(value -> value.toUpperCase());

// map: transform BOTH key and value, 1-to-1 — may trigger repartitioning downstream
KStream<String, Integer> lengths = stream.map((key, value) ->
        KeyValue.pair(key, value.length()));
```

PREFER MAPVALUES WHEN POSSIBLE

Because `mapValues` never changes the key, Kafka Streams knows the existing partitioning is still valid — no repartition topic is created. Use `map` only when you actually need to change the key.

3.5 Filter / FilterNot

```
KStream<String, String> longWordsOnly = stream.filter((key, value) -> value.length() > 5);
KStream<String, String> shortWordsOnly = stream.filterNot((key, value) -> value.length() > 5);
```

3.6 FlatMapValues / FlatMap

```
// flatMapValues: one input record -> zero or more output records, same key
KStream<String, String> words = stream.flatMapValues(line -> Arrays.asList(line.split(" ")));

// flatMap: one input record -> zero or more (key, value) pairs, key can change too
KStream<String, String> wordsWithNewKey = stream.flatMap((key, line) -> {
    List<KeyValue<String, String>> result = new ArrayList<>();
    for (String word : line.split(" ")) {
        result.add(KeyValue.pair(word, word));
    }
    return result;
});
```

3.7 Branch

Splits a single KStream into multiple KStreams based on predicates, evaluated in order (each record goes to the *first* matching branch only, mirroring an if/else-if chain). In modern Kafka Streams (2.8+) this is exposed via `split()`:

```
Map<String, KStream<String, String>> branches = stream.split(Named.as("branch-"))
    .branch((key, value) -> value.startsWith("A"), Branched.as("starts-with-a"))
    .branch((key, value) -> value.startsWith("B"), Branched.as("starts-with-b"))
    .defaultBranch(Branched.as("everything-else"));

KStream<String, String> startsWithA = branches.get("branch-starts-with-a");
```

3.8 SelectKey

```
// re-key a stream, e.g. so a downstream groupByKey groups correctly
KStream<String, String> rekeyed = stream.selectKey((key, value) -> value.substring(0, 1));
```

REPARTITIONING TRIGGER

`selectKey`, `map`, and `flatMap` can all change the record key. Any key change downstream of a stateful operation (groupBy, join, aggregate) forces Kafka Streams to insert an internal repartition topic — see Section 3.9.

3.9 Reading from Kafka

```
KStream<String, String> stream = builder.stream("input-topic");
KTable<String, String>  table  = builder.table("input-topic");
GlobalKTable<String, String> globalTable = builder.globalTable("input-topic");
```

3.10 Writing to Kafka

```
stream.to("output-topic", Produced.with(Serdes.String(), Serdes.String()));

// or, if you need the resulting KStream/KTable for further chaining:
KStream<String, String> sameStream = stream.through("intermediate-topic");
```

3.11 Streams Marked for Re-Partition

Whenever the DSL detects that the key may have changed and the next operator requires correct partitioning (a join, groupBy, or aggregate), it automatically inserts a hidden repartition topic: data is written out, then re-read, guaranteeing all records for a given key land on the same partition/task before the stateful step runs.

```
KStream (key changed by map/selectKey)
        |
        ▼  Kafka Streams detects downstream groupByKey/join
        |
        ▼  auto-creates <app-id>-XXX-repartition topic
        |         (produced, then re-consumed)
        ▼
groupByKey() / join() — now guaranteed correctly partitioned
```

PERFORMANCE NOTE

Repartitioning means an extra produce + consume round trip for every affected record. It's necessary for correctness, but it isn't free — avoid unnecessary key changes right before stateful operators when you can restructure the topology to avoid them.

3.12 Refresher on Log Compaction

Kafka topics backing KTables and changelog topics typically use **log compaction** (`cleanup.policy=compact`) rather than time/size-based deletion. Compaction guarantees that, for each key, at least the last known value is retained forever — perfect for representing "current state" rather than a full history. A record with a `null` value acts as a **tombstone**, and the compactor will eventually remove that key entirely.

3.13 KStream and KTable Duality

A KTable can always be thought of as the "compacted view" of a KStream, and a KStream can always be derived from a KTable by treating every update as a fact: this is the **stream-table duality**. It is the conceptual foundation that makes conversions between the two possible in both directions.

3.14 Transforming a KTable to a KStream

```
KStream<String, Long> asStream = wordCountsTable.toStream();
```

3.15 Transforming a KStream to a KTable

```
// via an explicit intermediate topic + table read
stream.to("intermediate-topic");
KTable<String, String> asTable = builder.table("intermediate-topic");

// or, more directly, by grouping and reducing/aggregating (see Chapter 4)
KTable<String, String> latestPerKey = stream
        .groupByKey()
        .reduce((oldValue, newValue) -> newValue);
```

3.16 Section Summary

Chapter 3 Takeaways

- KStream = independent facts; KTable = latest-value-per-key (with tombstone support via null values).
- Stateless ops (map, filter, flatMap, branch, selectKey) need no state store and are cheap.
- Changing a key can silently trigger a repartition topic — know which operators can do this.
- Log compaction is what makes KTables and changelog topics behave like durable "current state" tables.
- KStream ↔ KTable conversions are always possible thanks to stream-table duality.

KStreams and KTables — Advanced Operations (Stateful)

4.1 Section Objective

Cover the operators that require a local state store: `groupBy`, `count`, `aggregate`, `reduce`, plus the utility operators `peek` and `transform` / `transformValues`, and how to write results to a non-Kafka system.

4.2 KTable groupBy

Just like a KStream, a KTable can be re-grouped by a different key in preparation for a downstream aggregation — for example, re-aggregating per-user data into per-region totals:

```
KGroupedTable<String, String> regrouped = usersTable
        .groupBy((key, value) -> KeyValue.pair(extractRegion(value), value));
```

4.3 KGroupedStream / KGroupedTable — Count

```
KTable<String, Long> counts = groupedStream.count(Materialized.as("counts-store"));
```

`count()` is really a special case of `aggregate()` where the aggregator simply increments a running `Long` for every incoming record on that key.

4.4 KGroupedStream / KGroupedTable — Aggregate

`aggregate()` is the general-purpose stateful operator: you supply an initializer (the starting value, e.g. an empty object) and an aggregator function (how to fold a new record into the running result):

```
KTable<String, Long> totalCharacters = groupedStream.aggregate(
    () -> 0L,                                          // initializer
    (key, newValue, aggValue) -> aggValue + newValue.length(), // aggregator
    Materialized.<String, Long, KeyValueStore<Bytes, byte[]>>as("chars-store")
        .withValueSerde(Serdes.Long())
);
```

WHEN TO REACH FOR AGGREGATE()

Use `aggregate()` whenever the result type differs from the input type — e.g. counting occurrences (Long from String), building a running list, or computing a running average (which needs both a sum and a count packed into a custom object).

4.5 KGroupedStream / KGroupedTable — Reduce

`reduce()` is a simpler variant of `aggregate()` for the common case where the output type is the **same** as the input type — no separate initializer is needed because the first record for a key simply becomes the initial value:

```
KTable<String, Long> runningSum = groupedStream.reduce((aggValue, newValue) -> aggValue + newValue);

// "keep the latest value seen per key" is a very common reduce idiom:
KTable<String, String> latestPerKey = groupedStream.reduce((oldValue, newValue) -> newValue);
```

Operator	Input type == Output type?	Needs initializer?	Typical use
`count()`	No (always Long)	No (implicit 0)	Occurrence counting
`reduce()`	Yes, required	No (first value is the seed)	Running sum, latest-value, min/max
`aggregate()`	No, can differ	Yes, required	Anything else — running averages, custom accumulator objects

4.6 KStream peek

`peek()` lets you observe records flowing through the topology (e.g. for logging or metrics) without altering the stream at all — it always returns the exact same KStream unchanged:

```
stream.peek((key, value) -> log.info("Observed record: key={} value={}", key, value))
      .filter((key, value) -> value != null);
```

4.7 KStream Transform / TransformValues

`transform` / `transformValues` drop down closer to the Processor API from within the DSL, granting access to a state store and to the

`ProcessorContext` (e.g. for scheduling periodic `punctuate` callbacks) — useful when the built-in DSL operators aren't expressive enough:

```
KStream<String, String> transformed = stream.transformValues(
    () -> new ValueTransformerWithKey<String, String, String>() {
        private KeyValueStore<String, Long> store;

        @Override
        public void init(ProcessorContext context) {
            store = (KeyValueStore<String, Long>) context.getStateStore("my-store");
        }

        @Override
        public String transform(String key, String value) {
            Long previousCount = store.get(key);
            long newCount = (previousCount == null ? 0 : previousCount) + 1;
            store.put(key, newCount);
            return value + " (seen " + newCount + " times)";
        }

        @Override
        public void close() { }
    },
    "my-store"
);
```

TRANSFORM VS. MAP/MAPVALUES

Only reach for `transform` / `transformValues` when you genuinely need a state store or scheduled callbacks inside a single step — for everything else, the plain DSL operators (map, filter, aggregate) are simpler and less error-prone.

4.8 What if I Want to Write to an External System?

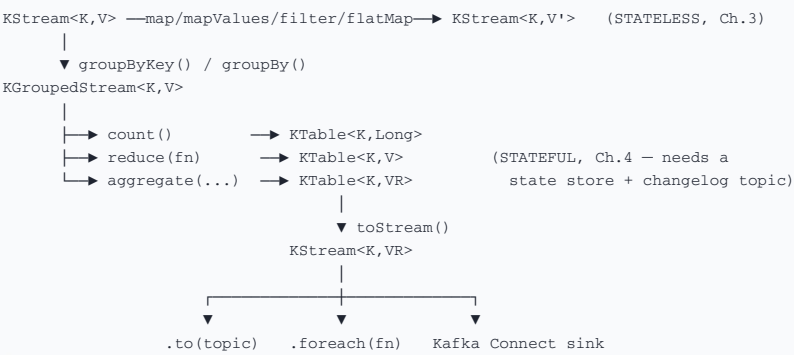
Not every pipeline ends by writing back to a Kafka topic. Two common patterns:

1. **foreach()** — a terminal operation for side-effecting on every record (e.g. calling a REST API or writing to a database). Because it's outside Kafka's transactional guarantees, delivery is inherently at-least-once from the external system's point of view unless you build your own idempotency there.

   ```
   stream.foreach((key, value) -> externalDatabase.upsert(key, value));
   ```

2. **Kafka Connect** — the generally recommended approach: write your Streams output to a plain Kafka topic, then use a Kafka Connect sink connector (JDBC, Elasticsearch, S3, etc.) to deliver it to the external system, keeping your Streams app simple and decoupling delivery concerns.

4.9 Summary Diagram



Chapter 4 Takeaways

- `count` ⊂ `reduce` ⊂ `aggregate` in terms of generality — pick the simplest one that fits.
- Every stateful operator needs a named state store, which is automatically backed by a changelog topic.
- `peek` never mutates the stream; use it purely for observation/logging.
- `transform` / `transformValues` unlock direct state-store access when the DSL alone isn't enough.
- For external sinks, prefer Kafka Connect over ad-hoc `foreach()` calls when delivery guarantees matter.

CHAPTER 5

Exactly-Once Semantics — Theory

5.1 What's Exactly-Once?

When a system processes data, there are three possible delivery/processing guarantees:

Guarantee	Meaning	Risk
At-most-once	A record is processed 0 or 1 times	Data loss on failure
At-least-once	A record is processed 1 or more times	Duplicate processing / double-counting on failure
Exactly-once (EOS)	A record is processed *as if* exactly once, even in the presence of failures and retries	Requires transactional coordination — some throughput cost

"Exactly once" doesn't mean the bytes physically touch the network only one time — retries still happen. It means the *observable effect* (the committed output and offset) behaves as if it happened exactly once, with no visible duplicates and no visible gaps.

5.2 Exactly-Once in Kafka 0.11

Kafka introduced the building blocks for exactly-once in version 0.11 (2017). It's worth being precise about what is actually guaranteed: **Kafka does not guarantee that a message is sent over the network only once.** Retries at the TCP/producer level still happen exactly as they always have. What Kafka guarantees instead is that the *final state* of your system — the committed output topic and the committed consumer offset — is updated exactly once, which makes any underlying network-level duplicates harmless. This is achieved with two composable features:

5.2.1 Idempotent Producer (the foundation)

This solves duplicates on the **Producer → Broker** hop specifically.

> **THE PROBLEM IT SOLVES**
> A producer sends a message, does not receive an ACK in time (network blip, broker GC pause, etc.), and retries — writing the same logical message to the partition a second time.

The fix: enable `enable.idempotence=true`. The broker assigns the producer a **PID** (Producer ID) and tracks a **sequence number** per partition for every message that producer sends.

- **The check:** when the broker receives a message, it compares the incoming sequence number to the last committed sequence number for that (PID, partition) pair.
- If the incoming sequence number is **lower than or equal to** the last committed one, the broker silently rejects it as a duplicate — the producer sees a successful ACK either way, so retries are always safe.

5.2.2 Transactions (atomic writes)

Idempotence alone only protects a single partition against producer retries. It does *not* solve the harder problem: atomically writing to **multiple partitions/topics** together with committing a **consumer offset** — the exact shape of a "consume → process → produce" pipeline.

The goal: if you consume from Topic A, process a record, and produce a result to Topic B, either *both* the new message on Topic B and the consumer-offset commit for Topic A succeed, or *neither* does. There is no in-between state where the output is written but the offset isn't (or vice versa).

The mechanism:

- The producer is configured with a `transactional.id` — a stable identifier across restarts.
- A **Transaction Coordinator** (a designated broker) manages the internal `__transaction_state` log for that `transactional.id`.
- The producer calls, in order: `initTransactions()` once at startup, then `beginTransaction()`, its normal `send()` calls, and finally `commitTransaction()` (or `abortTransaction()` on failure).
- Crucially, the **consumer** on the reading side must set `isolation.level=read_committed`. This makes it skip any messages that belong to a transaction that hasn't committed yet (or that was aborted) — it will never observe partial, in-flight transactional writes.

> **THE GOLDEN RULE**
> To get true exactly-once semantics in a consumer→process→producer style application (which is exactly what Kafka Streams does internally), you must:
>
> 1. Use an idempotent producer (`enable.idempotence=true`).
> 2. Set `acks=all` (a hard requirement for idempotence to be honored).
> 3. Commit the consumer offset **inside the same transaction** as the produced output record(s).

Kafka Streams builds directly on top of these two primitives: each stream task's "consume → process → produce → commit offsets" cycle can be wrapped in a single Kafka transaction, which is exactly what happens when you set `PROCESSING_GUARANTEE_CONFIG = EXACTLY_ONCE_V2` (Section 5.4).

5.3 What's the Problem With At-Least-Once Anyway?

Consider our WordCount application under default (at-least-once) settings. Processing works like this per task: *(1)* consume a batch of records, *(2)* update local state (increment counts), *(3)* produce results, *(4)* commit consumer offsets. If the application crashes **after step 3 but before step 4**, on restart it will re-consume the same batch of records (since offsets weren't committed) — and increment the same counts a second time. The output topic now shows an inflated, incorrect count.

```
At-Least-Once failure window:

consume batch ──► update state ──► produce output ──► [CRASH HERE] ──► commit offsets
                                          ▲
                                          └── output already visible to downstream consumers,
                                              but offsets never committed
                                              ⇒ batch re-processed on restart ⇒ DOUBLE COUNT
```

For use cases like word counts this might be a minor annoyance; for use cases like the BankBalance exercise in Chapter 6 — where an incorrect duplicate could mean a customer's balance is wrong — it's unacceptable.

5.4 How to Do Exactly-Once in Kafka Streams

Kafka Streams makes EOS a single configuration line — no application code changes required:

```
// Kafka 2.6+
config.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsConfig.EXACTLY_ONCE_V2);

// older Kafka versions (2.5 and below) used:
// config.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsConfig.EXACTLY_ONCE);
```

With this flag set, Kafka Streams wraps each task's consume-process-produce-commit cycle in a Kafka transaction: the produced output records and the consumer offset commit either both become visible together, or neither does. On restart after a crash, there is no window where output was produced but offsets weren't committed — so no double counting.

DOWNSTREAM CONSUMERS MUST COOPERATE

Any consumer reading the output of an EOS Kafka Streams application must set `isolation.level=read_committed` to actually see the exactly-once guarantee — otherwise it may read uncommitted (later aborted) records from in-flight transactions.

EOS IS NOT FREE

Transactional writes add coordination overhead (transaction markers, commit latency), so exactly-once processing trades some throughput/latency for correctness. Enable it when correctness genuinely requires it — financial balances, inventory counts, billing — not by default for every pipeline.

5.5 Kafka CLI Hands-On (Setup & Verification)

Before writing any application code, it's worth configuring your brokers correctly and exercising the transactional machinery directly from the command line so the moving parts feel concrete.

5.5.1 Broker Configuration (Required)

Add these to `server.properties` on every broker (whether you're running Zookeeper-mode or KRaft-mode) and restart:

```
# Minimum required for transactions (3 brokers recommended in production)
transaction.state.log.replication.factor=3
transaction.state.log.min.isr=2

# Allow transactional producers to keep their transactional.id reserved
transactional.id.expiration.ms=604800000
```

SINGLE-BROKER DEV CLUSTERS

On a local single-broker cluster you won't be able to satisfy a replication factor of 3 — drop `transaction.state.log.replication.factor` to `1` and `transaction.state.log.min.isr` to `1` for local testing only. Never do this in production: it removes the durability transactions are meant to provide.

5.5.2 Create a Transactional Producer via CLI

You cannot run a full consume-transform-produce transaction purely from the CLI, but you *can* exercise idempotence end-to-end:

```
# Create a test topic
kafka-topics.sh --bootstrap-server localhost:9092 --create --topic eos-test \
  --partitions 1 --replication-factor 1

# Start a console consumer reading only committed messages
kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic eos-test \
  --isolation-level read_committed \
  --from-beginning

# Start a producer with idempotence enabled
kafka-console-producer.sh --bootstrap-server localhost:9092 \
  --topic eos-test \
  --producer-property enable.idempotence=true \
  --producer-property acks=all
```

> **NOTE**
> If you type messages into this producer, you'll see them appear in the consumer instantly — the console producer commits each message essentially immediately, so there's no visible "in-flight" delay to observe here. The real value of this lab is confirming your CLI tooling and broker configuration accept idempotent, `acks=all` producers and `read_committed` consumers without error before you introduce application code.

5.5.3 Monitoring the Transaction Log

Kafka stores all transaction state internally, in the `__transaction_state` topic. You can inspect the raw segment files directly:

```
# Dump the internal transaction state log to see active/committed transactions
kafka-dump-log.sh --files /tmp/kraft-combined-logs/__transaction_state-0/*.log --print-data-log
```

This surfaces the `PID`, the `transactional.id`, and the current **epoch** (used for fencing — see the pitfalls list in Section 5.7) for every transaction the coordinator has tracked.

5.6 Spring Boot Java Hands-On

This section walks through a realistic "Consume-Process-Produce" pattern with full exactly-once guarantees, using Spring Kafka's transaction integration rather than the raw `KafkaProducer` / `KafkaConsumer` transaction API.

5.6.1 Dependencies (Maven)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

5.6.2 Producer Configuration (the transactional producer)

```
@Configuration
public class KafkaProducerConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Bean
    public ProducerFactory<String, String> producerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);

        // 1. Enable EOS — this pairs with acks=all and unlimited retries
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5); // safe with idempotence

        // 2. Crucial: set a transactional ID, unique per producer instance
        // In production, derive this from a stable identifier (e.g., hostname + app name).
        props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "my-app-txn-id-1");

        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        return new DefaultKafkaProducerFactory<>(props);
    }

    @Bean
    public KafkaTransactionManager<String, String> kafkaTransactionManager(
            ProducerFactory<String, String> producerFactory) {
        // Binds Kafka transactions to Spring's @Transactional
        return new KafkaTransactionManager<>(producerFactory);
    }
}
```

5.6.3 Consumer Configuration (read_committed)

```
@Configuration
public class KafkaConsumerConfig {

    @Bean
    public ConsumerFactory<String, String> consumerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "eos-group");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);

        // 3. Must be read_committed to skip uncommitted (aborted/in-flight) messages
        props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");

        // 4. Disable auto-commit — offsets are committed inside the transaction instead
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
        return new DefaultKafkaConsumerFactory<>(props);
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, String> kafkaListenerContainerFactory(
            ConsumerFactory<String, String> consumerFactory,
            KafkaTransactionManager<String, String> transactionManager) {

        ConcurrentKafkaListenerContainerFactory<String, String> factory =
                new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory);

        // 5. Bind the transaction manager to the listener container itself
        factory.getContainerProperties().setTransactionManager(transactionManager);
        factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.RECORD);
        return factory;
    }
}
```

5.6.4 The Service Layer (the exactly-once logic)

This is where it all comes together. Note the `@Transactional` annotation, explicitly pointed at our Kafka transaction manager:

```
@Service
public class OrderProcessor {

    private final KafkaTemplate<String, String> kafkaTemplate;

    public OrderProcessor(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "input-orders", groupId = "eos-group")
    @Transactional(transactionManager = "kafkaTransactionManager")
    public void process(String orderId, String orderDetails) {

        // 1. Consume message (handled automatically by the listener).
        System.out.println("Processing: " + orderDetails);

        // 2. Business logic (e.g., enrich, transform)
        String processedPayload = "PROCESSED_" + orderDetails.toUpperCase();

        // 3. Produce the result — this send() is now PART OF THE TRANSACTION
        kafkaTemplate.send("output-orders", orderId, processedPayload);

        // 4. The consumer offset for this record commits inside the SAME
        //    transaction as the send() above. If this method throws,
        //    the whole transaction aborts: no output is written, and the
        //    offset is not committed — the input message will be redelivered.
    }
}
```

5.6.5 Initializing the Producer (crucial step)

A transactional producer must call `initTransactions()` once, before its first transaction, or the very first `@Transactional` method invocation will fail:

```
@Component
public class TransactionInitializer {

    @EventListener(ApplicationReadyEvent.class)
    public void init(KafkaTemplate<String, String> kafkaTemplate) {
        // "Wakes up" the transactional producer and registers it with the coordinator.
        // Without this, your first transaction will throw an error.
        kafkaTemplate.getProducerFactory().createProducer().initTransactions();
        System.out.println("Transactional Producer Initialized.");
    }
}
```

5.6.6 Summary of the Execution Flow

Step	Actor	Action	Guarantee
1	Consumer	Reads Order A from `input-orders`.	In-memory only — the offset is not committed yet.
2	Processor	Runs business logic.	No external writes yet.
3	Producer	Sends Order A to `output-orders`.	Message is staged (uncommitted). A `read_uncommitted` consumer could see it, but our `read_committed` consumer cannot.
4	Transaction Manager	Commits the transaction.	Atomic: the output topic gets the message *and* the consumer offset moves to the next record together. If the broker dies here, nothing is committed.
5	Consumer (downstream app)	Reads from `output-orders`.	Because `isolation.level=read_committed`, it only sees Order A now — never a partially-written duplicate.

5.7 Common Pitfalls to Avoid

- Do **not** leave `auto.commit=true` on your consumer. Spring may override it in some configurations, but set `ENABLE_AUTO_COMMIT_CONFIG=false` explicitly rather than relying on that.
- Do **not** mix in Spring ORM's `@Transactional` (e.g. `JpaTransactionManager`) on the same method by accident. Always be explicit: `@Transactional(transactionManager = "kafkaTransactionManager")`.
- Idempotence guarantees can break if `MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION` is set above `5`, unless you are running a broker/client version (2.0+) that added support for larger in-flight windows with idempotence — when in doubt, keep it at `5` or lower.
- Ensure every application **instance** uses a unique `transactional.id`. If two instances share the same `transactional.id`, the broker will **fence** (kill) the older producer the moment the newer one calls `initTransactions()` — by design, to prevent two producers from writing conflicting data under the same transactional identity.

Chapter 5 Takeaways

- EOS means results behave as if each record was processed exactly once, despite retries/crashes — achieved by making duplicates *harmless*, not by eliminating network-level retries.
- The idempotent producer solves Producer→Broker duplicates via a PID + per-partition sequence number check.
- Transactions solve the harder multi-partition + consumer-offset problem, coordinated by a Transaction Coordinator broker and a `transactional.id`.
- At-least-once can silently double-process a batch if a crash happens between "produce" and "commit offsets."
- In Kafka Streams, EOS is one config line: `PROCESSING_GUARANTEE_CONFIG = EXACTLY_ONCE_V2`; in a hand-rolled Spring Kafka app, it takes an idempotent+transactional producer, a `read_committed` consumer, and a shared `KafkaTransactionManager`.
- A unique `transactional.id` per instance is not optional — reusing one gets your older producer fenced off.

Exactly-Once Semantics — Practice Exercise: BankBalance

6.1 BankBalance — Exercise Overview

This hands-on exercise puts EOS theory into practice by building a small pipeline that would be genuinely broken by duplicate processing: a running bank account balance.

GOAL

Produce a stream of random bank `transaction` events (name, amount, time), then build a Kafka Streams application that maintains a running, correct `{ balance, transaction-count }` JSON object per account — and prove it stays correct even under simulated failure/restart, thanks to exactly-once semantics.

Data model

```
// input record value — one JSON object per transaction
{
  "name": "john",
  "amount": 123,
  "time": "2024-01-01T12:00:00.000Z"
}

// output record value — the running balance, keyed by "name"
{
  "count": 4,
  "balance": 456,
  "time": "2024-01-01T12:00:03.000Z"
}
```

6.2 Kafka Producer Guidance

Build a small standalone producer application that:

- Randomly picks a name from a small fixed pool (e.g. `john`, `stephane`, `alice`).
- Generates a random transaction amount (can be positive or negative, to simulate deposits and withdrawals).
- Serializes the record to JSON and sends it, keyed by the account name, to a topic `bank-transactions`.
- Sends records in a loop with a short random delay, so you have a continuous stream to observe.

6.3 Kafka Producer Solution

```
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
// idempotent producer — good practice even outside a Streams transaction
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
String[] names = {"john", "stephane", "alice", "bob"};
Random random = new Random();

while (true) {
    String name = names[random.nextInt(names.length)];
    int amount = random.nextInt(1000) - 500; // -500..+499
    String time = Instant.now().toString();

    String json = String.format(
        "{\"name\":\"%s\",\"amount\":%d,\"time\":\"%s\"}", name, amount, time);

    producer.send(new ProducerRecord<>("bank-transactions", name, json),
        (metadata, exception) -> {
            if (exception != null) exception.printStackTrace();
        });

    Thread.sleep(random.nextInt(500));
}
```

6.4 Kafka Streams Guidance & Hints

- Read `bank-transactions` as a `KStream<String, String>` (raw JSON string values).
- `groupByKey()` — the record key is already the account name, so no `selectKey`/repartition is needed here.

- Use `aggregate()`, not `reduce()`: the accumulated value's shape (`{count, balance, time}`) differs from the input's shape (`{name, amount, time}`).
- The initializer should produce a starting JSON object with `balance = 0` and `count = 0`.
- Materialize the resulting KTable so it's queryable and durable, then push it to an output topic via `toStream().to(...)`.
- Enable `EXACTLY_ONCE_V2` in the Streams config — this is the whole point of the exercise.

6.5 Kafka Streams Solution

```
Properties config = new Properties();
config.put(StreamsConfig.APPLICATION_ID_CONFIG, "bank-balance-application");
config.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
config.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass());
config.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.String().getClass());
config.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsConfig.EXACTLY_ONCE_V2);

StreamsBuilder builder = new StreamsBuilder();
KStream<String, String> transactions = builder.stream("bank-transactions");

KTable<String, String> bankBalance = transactions.groupByKey().aggregate(
    () -> "{\"count\":0,\"balance\":0,\"time\":null}",
    (name, transactionJson, balanceJson) -> {
        JsonObject tx = JsonParser.parseString(transactionJson).getAsJsonObject();
        JsonObject bal = JsonParser.parseString(balanceJson).getAsJsonObject();

        int newCount = bal.get("count").getAsInt() + 1;
        int newBalance = bal.get("balance").getAsInt() + tx.get("amount").getAsInt();
        String newTime = tx.get("time").getAsString();

        JsonObject result = new JsonObject();
        result.addProperty("count", newCount);
        result.addProperty("balance", newBalance);
        result.addProperty("time", newTime);
        return result.toString();
    },
    Materialized.<String, String, KeyValueStore<Bytes, byte[]>>as("bank-balance-store")
        .withKeySerde(Serdes.String())
        .withValueSerde(Serdes.String())
);

bankBalance.toStream().to("bank-balance-output", Produced.with(Serdes.String(), Serdes.String()));

KafkaStreams streams = new KafkaStreams(builder.build(), config);
streams.start();
Runtime.getRuntime().addShutdownHook(new Thread(streams::close));
```

LIBRARY NOTE

The solution above uses Google's `gson` for JSON parsing purely for readability — any JSON library (Jackson, org.json) works equally well; the DSL logic is identical regardless of library choice.

6.6 Running the BankBalance Application

Hands-On Lab 6.1 — Prove Exactly-Once Works

```
# create topics
kafka-topics.sh --create --topic bank-transactions --bootstrap-server localhost:9092 --partitions 3
kafka-topics.sh --create --topic bank-balance-output --bootstrap-server localhost:9092 \
  --partitions 3 --config cleanup.policy=compact

# run the producer, then the streams app
java -jar bank-producer.jar
java -jar bank-streams-app.jar

# watch the running balance per account, read with read_committed isolation
kafka-console-consumer.sh --topic bank-balance-output --bootstrap-server localhost:9092 \
  --from-beginning --property print.key=true --isolation-level read_committed
```

1. Let the pipeline run for a few seconds so balances start accumulating.
2. Kill the Streams application mid-flight (`Ctrl+C` or `kill -9` to simulate an ungraceful crash) while the producer keeps running.
3. Restart the Streams application.
4. Confirm each account's `count` matches the number of transactions actually sent for it — no jumps, no double counting — verifying the exactly-once guarantee held even through the crash.

6.7 Section Summary

Chapter 6 Takeaways

- BankBalance is a realistic case where at-least-once duplicates would produce visibly wrong results (incorrect balances).
- `aggregate()` is the right tool whenever the running-state shape differs from the input record shape.
- Turning on `EXACTLY_ONCE_V2` is enough to keep the running balance correct across ungraceful restarts.
- Downstream consumers must set `isolation.level=read_committed` to see the benefit.

CHAPTER 7

Joins — KStream to GlobalKTable & KTable to GlobalKTable

7.1 What Are Joins in Kafka Streams?

Joins let you combine two streams/tables by key, in real time, without a database — for example enriching a stream of raw page-view events with user profile data as each event arrives. Kafka Streams supports several join shapes, summarized below.

7.2 Join Constraints and GlobalKTables

Ordinary joins (KStream-KStream, KStream-KTable, KTable-KTable) require both sides to be **co-partitioned**: same number of partitions, same keying scheme, so that a given key's data always lands on the same partition on both sides and can be joined locally within a task, without shuffling data across the network.

A **GlobalKTable** relaxes this constraint: it loads the *entire* table into every single application instance (not just the instance's assigned partitions). This means a KStream can join against a GlobalKTable with **no co-partitioning requirement at all** — useful exactly for reference/lookup data like a user directory, where the input stream's key (e.g. `userId`) doesn't need to match the table's partitioning.

	KTable	GlobalKTable
Data distribution	Partitioned — each instance holds only its assigned partitions	Fully replicated — every instance holds the entire table
Co-partitioning required to join?	Yes	No
Good for	Large, frequently-updated tables already keyed the same as the stream	Smaller reference/lookup tables (e.g. user profiles, product catalog)
Memory footprint	Lower per-instance (only a partition's worth)	Higher per-instance (the whole table, everywhere)

7.3 The Different Types of Joins: Inner, Left, Outer

Join type	Behavior
Inner join	Emits a result only when a matching record exists on *both* sides for that key.
Left join	Always emits a result for every left-side record; the right side is included if a match exists, otherwise it's passed as `null`.
Outer join	(KStream-KStream and KTable-KTable only) Emits a result whenever *either* side has a new record, filling in `null` for whichever side has no current match. Not applicable to KStream-GlobalKTable joins.

GLOBALKTABLE JOIN LIMITATION

KStream-GlobalKTable joins only support **inner** and **left** joins (the GlobalKTable can never be the "driving" side that triggers new output on its own updates — only the KStream side triggers a join lookup).

7.4 Creating a Join with the UserEnrich Application

Scenario: a stream of raw `page-views` keyed by `userId`, and a reference `user-profiles` table (also keyed by `userId`) that changes independently and infrequently. We enrich each page view with the user's region, using a GlobalKTable so we don't need to worry about co-partitioning.

```
StreamsBuilder builder = new StreamsBuilder();

KStream<String, String> pageViews = builder.stream("page-views");
GlobalKTable<String, String> userProfiles = builder.globalTable("user-profiles");

KStream<String, String> enrichedViews = pageViews.leftJoin(
    userProfiles,
    (viewKey, viewValue) -> viewKey,          // key extractor: how to look up into the GlobalKTable
    (view, profile) -> {
        String region = (profile != null) ? extractRegion(profile) : "UNKNOWN";
        return view + ",region=" + region;
    }
);

enrichedViews.to("page-views-enriched");
```

WHY LEFTJOIN HERE

We use `leftJoin` because we always want to emit an enriched page view even if the user profile hasn't arrived yet or doesn't exist — falling back to `"UNKNOWN"` rather than silently dropping the page-view event, which an inner join would do.

7.5 All the Joins Explained

Beyond the KStream-GlobalKTable example above, it's worth knowing the full join matrix conceptually:

Left side	Right side	Supported join types	Windowed?
KStream	KStream	Inner, Left, Outer	Yes — requires a `JoinWindows` time bound, since two independent streams need a time frame to be considered "concurrent"
KStream	KTable	Inner, Left	No — always joins against the table's current (latest) value
KStream	GlobalKTable	Inner, Left	No
KTable	KTable	Inner, Left, Outer	No — always current-value-to-current-value

7.7 KTable to GlobalKTable — the Fastest Join in Kafka Streams

Beyond the KStream-GlobalKTable pattern above, a **KTable-to-GlobalKTable** join deserves its own walkthrough — it's one of the most powerful, and fastest, joins available in the DSL. The golden rule that makes it special:

- A **KTable** is partitioned by key across your app instances — only some keys live on each instance.
- A **GlobalKTable** is fully replicated across all instances — every instance holds the entire dataset.

Because the GlobalKTable lives everywhere, this join **never requires a network shuffle or repartitioning**. It is a lightning-fast, purely local lookup — no matter how many application instances you're running.

7.7.1 The Scenario

Source	Key	Value
KTable `orders`	`orderId`	`{ userId, product, amount }`
GlobalKTable `users`	`userId`	`{ name, email }`

We want to enrich each order with the user's name. Notice the keys **do not match** (`orderId` vs. `userId`) — so this requires a **foreign-key join** via a `KeyValueMapper`.

7.7.2 The Data Model (POJOs)

```
public class Order {
    private String userId;
    private String product;
    private double amount;
    // getters, setters, constructors...
}

public class User {
    private String name;
    private String email;
    // getters, setters, constructors...
}

public class EnrichedOrder {
    private String orderId;
    private String product;
    private double amount;
    private String userName;
    private String userEmail;
    // getters, setters, constructors...
}
```

7.7.3 The Topology (the Join Logic)

```java
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.GlobalKTable;
import org.apache.kafka.streams.kstream.KTable;
import org.apache.kafka.streams.kstream.KeyValueMapper;
import org.apache.kafka.streams.kstream.ValueJoiner;
import org.apache.kafka.streams.kstream.Materialized;

public class EcommerceTopology {

    public static void build(StreamsBuilder builder) {

        // 1. Source KTable (partitioned normally)
        KTable<String, Order> ordersTable = builder.table(
            "orders-topic",
            Materialized.as("orders-store") // local state store
        );

        // 2. Source GlobalKTable (replicated everywhere)
        GlobalKTable<String, User> usersTable = builder.globalTable(
            "users-topic",
            Materialized.as("users-global-store")
        );

        // 3. The foreign-key join (KTable -> GlobalKTable)
        KTable<String, EnrichedOrder> enrichedOrders = ordersTable.join(
            usersTable,
            // KeyValueMapper:
            // Input: (orderId, Order). Output: the lookup key for the GlobalKTable (userId).
            // This tells Kafka: "To join this order, go find the User with this userId."
            new KeyValueMapper<String, Order, String>() {
                @Override
                public String apply(String orderId, Order order) {
                    return order.getUserId(); // extract the foreign key
                }
            },
            // ValueJoiner:
            // Input: (Order, User). Output: the final combined object.
            new ValueJoiner<Order, User, EnrichedOrder>() {
                @Override
                public EnrichedOrder apply(Order order, User user) {
                    return new EnrichedOrder(
                        order.getOrderId(),
                        order.getProduct(),
                        order.getAmount(),
                        user.getName(),      // enriched field
                        user.getEmail()      // enriched field
                    );
                }
            }
        );

        // 4. Send the result to an output topic
        enrichedOrders.toStream().to("enriched-orders-topic");
    }
}
```

Lambda style (Java 8+)

```java
KTable<String, EnrichedOrder> enrichedOrders = ordersTable.join(
    usersTable,
    (orderId, order) -> order.getUserId(),  // KeyValueMapper (lambda)
    (order, user) -> new EnrichedOrder(     // ValueJoiner (lambda)
        order.getOrderId(),
        order.getProduct(),
        order.getAmount(),
        user.getName(),
        user.getEmail()
    )
);
```

7.7.4 Spring Boot Integration

```
@Configuration
public class KafkaStreamsConfig {

    @Bean
    public StreamsBuilderFactoryBean streamsBuilderFactoryBean() {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "order-enricher-app");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass());
        props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, JsonSerde.class); // your JSON Serde

        // Kafka Streams automatically creates any needed internal topics.
        // For a GlobalKTable, make sure its source topic has enough partitions
        // for your throughput — no extra config is required for the join itself.

        StreamsBuilderFactoryBean factoryBean = new StreamsBuilderFactoryBean();
        factoryBean.setStreamsConfiguration(props);
        return factoryBean;
    }

    @Bean
    public KStream<String, EnrichedOrder> kStream(StreamsBuilderFactoryBean factoryBean) {
        StreamsBuilder builder = new StreamsBuilder();
        EcommerceTopology.build(builder);
        // the factoryBean builds and starts the topology automatically
        return null;
    }
}
```

7.8 Critical Behavior: What Happens on Updates?

This is the single most important thing to internalize about this join — understanding it up front prevents subtle bugs later:

Event	What Kafka Streams does
New Order arrives	Extracts the `userId`, looks up the User in the local GlobalKTable cache/store, and immediately outputs the enriched order.
Existing Order updates	Same as above — it re-joins with whatever User data is currently cached locally.
User updates (GlobalKTable)	Because every instance has the *full* users table, when a user's name changes, Kafka Streams automatically scans **all locally stored Orders** and finds every order that referenced that `userId` — then re-emits an updated `EnrichedOrder` for every matching order.
User does not exist yet	By default, `join()` performs an **inner join**. If no user is found, the order is dropped — no output is produced. Use `leftJoin()` to keep it.

WATCH YOUR ORDERS-TABLE SIZE

If your `orders` KTable is massive (millions of keys), updating a single User in the GlobalKTable triggers a full local scan of your orders store to find every matching order. This scan is efficient (pure in-memory), but on a very large table it can still spike CPU. Reserve this pattern for cases where the number of records referencing any single foreign key stays reasonably bounded, or where the GlobalKTable side updates infrequently.

7.9 Left Join vs. Inner Join (Foreign-Key Version)

To keep the order even when the user hasn't arrived yet (or is missing), switch to `leftJoin`:

```
KTable<String, EnrichedOrder> enrichedOrders = ordersTable.leftJoin(
    usersTable,
    (orderId, order) -> order.getUserId(),
    (order, user) -> {
        String name = (user == null) ? "UNKNOWN" : user.getName();
        String email = (user == null) ? "UNKNOWN" : user.getEmail();
        return new EnrichedOrder(order.getOrderId(), order.getProduct(), order.getAmount(), name, email);
    }
);
```

7.10 Alternative: Standard Join (When Keys Already Match)

If your KTable and GlobalKTable already share the exact same key, the `KeyValueMapper` can be skipped entirely:

```
// Assume ordersByUser has KEY = userId (not orderId)
KTable<String, Order> ordersByUser = builder.table("orders-by-user");
GlobalKTable<String, User> users = builder.globalTable("users");

// Simple join: same key (userId) matches directly, no mapper needed
KTable<String, EnrichedOrder> joined = ordersByUser.join(
    users,
    (order, user) -> new EnrichedOrder(user.getName(), order.getProduct())
);
```

7.11 Summary Cheatsheet — KTable to GlobalKTable

Aspect	KTable-to-GlobalKTable Join
Data distribution	KTable: partitioned. GlobalKTable: replicated.
Network shuffle	None — fully local lookup (fastest join shape in the DSL).
Required mapper	Yes, a `KeyValueMapper`, if it's a foreign-key join. Not needed if the keys are already identical.
Result type	A KTable — updates reactively as either side changes.
Memory cost	High: the GlobalKTable stores the entire source topic in memory on *every* instance.
Use case	Enriching fact tables (orders, transactions) with dimension tables (users, products, stores) that change relatively infrequently.

7.12 Running the Kafka Streams Join Application

🧪 Hands-On Lab 7.1 — UserEnrich

```
kafka-topics.sh --create --topic page-views --bootstrap-server localhost:9092 --partitions 3
kafka-topics.sh --create --topic user-profiles --bootstrap-server localhost:9092 --partitions 1
kafka-topics.sh --create --topic page-views-enriched --bootstrap-server localhost:9092 --partitions 3

# seed a couple of user profiles (compacted table)
kafka-console-producer.sh --topic user-profiles --bootstrap-server localhost:9092 \
  --property parse.key=true --property key.separator=,
> u1,{"name":"john","region":"EU"}
> u2,{"name":"alice","region":"US"}

# run the Streams app, then produce page views and observe enrichment
kafka-console-producer.sh --topic page-views --bootstrap-server localhost:9092 \
  --property parse.key=true --property key.separator=,
> u1,/home
> u3,/pricing   # no matching profile -> region=UNKNOWN

kafka-console-consumer.sh --topic page-views-enriched --bootstrap-server localhost:9092 --from-beginning
```

Chapter 7 Takeaways

- GlobalKTables trade memory (full replication on every instance) for freedom from co-partitioning constraints.
- Inner join = both sides must match; left join = left side always emitted, right side nullable; outer = either side triggers output.
- KStream-KStream joins need a time window; KStream-(Global)KTable joins don't — they always use the table's current value.
- GlobalKTable joins support inner/left only, never outer.
- KTable-to-GlobalKTable is the fastest join shape available: because the GlobalKTable is fully replicated, the join is a pure local lookup with zero network shuffle.
- Use a `KeyValueMapper` for foreign-key joins (keys differ, e.g. `orderId` vs. `userId`); skip it when both sides already share the same key.
- A GlobalKTable update triggers a local scan of the KTable side to re-emit every affected row — cheap in-memory, but worth watching on very large tables.

CHAPTER 8

Testing Your Kafka Streams Application

DISCLAIMER: ADVANCED LECTURES
This chapter assumes comfort with JUnit and Maven test scopes. If you're new to Java unit testing generally, review JUnit basics first — the Kafka-Streams-specific testing API itself is small and quick to pick up.

8.1 Testing in Kafka Streams

Kafka Streams ships a dedicated test-kit artifact — `kafka-streams-test-utils` — built around the `TopologyTestDriver` class. It lets you run your *real* topology definition entirely in-memory, with no broker, no Zookeeper, and no network calls, while still exercising the exact same DSL code path your production application uses.

```
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams-test-utils</artifactId>
    <version>3.7.0</version>
    <scope>test</scope>
</dependency>
```

8.2 Setup Your Kafka Streams Project (for Testing)

- Refactor your topology-building code into its own method (e.g. `Topology buildTopology()`) separate from `main()`, so both production code and tests can call it identically.
- Add the `kafka-streams-test-utils` dependency with `test` scope, as above.
- Create a standard JUnit test class under `src/test/java`.

8.3 Hands-On: Test Your WordCount Application

```
public class WordCountAppTest {
    private TopologyTestDriver testDriver;
    private TestInputTopic<String, String> inputTopic;
    private TestOutputTopic<String, Long> outputTopic;

    @BeforeEach
    public void setup() {
        Topology topology = WordCountApp.buildTopology();

        Properties config = new Properties();
        config.put(StreamsConfig.APPLICATION_ID_CONFIG, "wordcount-test");
        config.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "dummy:9092");
        config.put(StreamsConfig.STATE_DIR_CONFIG, "test-state-dir");

        testDriver = new TopologyTestDriver(topology, config);

        inputTopic = testDriver.createInputTopic(
            "streams-plaintext-input", new StringSerializer(), new StringSerializer());
        outputTopic = testDriver.createOutputTopic(
            "streams-wordcount-output", new StringDeserializer(), new LongDeserializer());
    }

    @AfterEach
    public void tearDown() {
        testDriver.close();
    }

    @Test
    public void testSingleLineProducesExpectedCounts() {
        inputTopic.pipeInput("kafka streams kafka");

        Map<String, Long> results = outputTopic.readKeyValuesToMap();

        assertEquals(2L, results.get("kafka"));
        assertEquals(1L, results.get("streams"));
    }

    @Test
    public void testRunningCountAccumulatesAcrossMultipleRecords() {
        inputTopic.pipeInput("hello world");
        inputTopic.pipeInput("hello kafka");

        List<KeyValue<String, Long>> outputs = outputTopic.readKeyValuesToList();

        // the LAST record for key "hello" should reflect the cumulative count of 2
        long lastHelloCount = outputs.stream()
            .filter(kv -> kv.key.equals("hello"))
            .reduce((first, second) -> second)
            .get().value;

        assertEquals(2L, lastHelloCount);
    }
}
```

WHY TOPOLOGYTESTDRIVER IS WORTH IT

Because it runs your actual production topology object, these tests catch real logic bugs (wrong split regex, off-by-one aggregation errors, wrong Serdes) with millisecond-fast, broker-free tests — perfect for CI pipelines.

8.4 Self-Guided Practice Exercises

📝 Try these on your own

- ☐ Write a `TopologyTestDriver` test for the BankBalance application (Chapter 6) that pipes in three transactions for the same account and asserts the final balance and count.
- ☐ Add a test case for a transaction stream containing *two different account keys* and assert both balances are tracked independently.
- ☐ Extend the UserEnrich test (Chapter 7) to cover the "no matching profile" case, asserting the output falls back to `region=UNKNOWN`.
- ☐ Add a negative test: pipe in malformed JSON and confirm your application either filters it out gracefully or routes it to a dead-letter topic, rather than throwing an unhandled exception.
- ☐ Measure test run time before and after switching from a real embedded broker (e.g. `EmbeddedKafkaCluster`) to `TopologyTestDriver`, and note the difference.

Chapter 8 Takeaways

- `TopologyTestDriver` runs your real topology in-memory — no broker required.

- Refactor topology-building into its own method so both `main()` and your tests share identical logic.
- `TestInputTopic` / `TestOutputTopic` give a clean, typed API for piping records in and reading results out.
- Fast, broker-free tests belong in CI for every stateful and stateless operator you rely on.

CHAPTER 9

KStream vs. KTable — The Big Idea in Plain English

9.1 The Big Idea (In Plain English)

Before diving back into code, it's worth restating the single most important mental model in Kafka Streams — in plain, jargon-free language:

- **KStream = a live feed.** Every single event that arrives is processed. If you send the same event twice, it processes it twice. It doesn't remember the past.
- **KTable = a snapshot of a database table.** It only cares about the latest value for each specific key. If you send an update for an existing key, the old value is thrown away and replaced.

> **KEEP THIS IN YOUR BACK POCKET**
> Every confusing behavior you'll ever hit with KStreams and KTables traces back to this one distinction: a KStream treats every record as a brand-new fact; a KTable treats every record as an update that overwrites whatever came before it for that key.

9.2 Simple Stateless Operations (No Memory Required)

These operations look at one event at a time and do not need to remember previous events.

Filter — keep only what you want

```
// KStream: Keep only orders over $100
stream.filter((key, price) -> price > 100);

// KTable: Keep only users whose status is "active"
table.filter((key, status) -> status.equals("active"));
```

Map — change the value or key

```
// Change the value: Convert the string to uppercase
stream.mapValues(value -> value.toUpperCase());

// Change both key and value
stream.map((key, value) -> KeyValue.pair(key + "_new", value.length()));
```

FlatMap — split one event into many

```
// Take one sentence and turn it into multiple words
stream.flatMapValues(sentence -> Arrays.asList(sentence.split(" ")));
```

9.3 Simple Stateful Operations (Requires Memory)

These operations must remember previous data to work (like counting or grouping).

Grouping and counting

```
// Group by key and count how many events per key
KGroupedStream grouped = stream.groupByKey();
KTable<String, Long> counts = grouped.count();
```

> **NOTE**
> If you do this on a KTable instead of a KStream, the count reflects how many times the key has been *updated*, not the final value itself.

Aggregating (sum, average, or combining values)

```
// Sum up all the prices per product ID
stream.groupByKey()
      .aggregate(() -> 0L, // start at zero
                 (key, newPrice, runningTotal) -> runningTotal + newPrice);
```

9.4 The Golden Rule: KStream vs. KTable in Operations

This is the most important difference to remember:

THE GOLDEN RULE

On a KStream: if you send 3 updates for the same customer ID, your `filter`, `map`, and `count` operations will run 3 times — each update is treated as an independent new event.

On a KTable: if you send 3 updates for the same customer ID, your `filter` and `map` operations will only ever reflect the latest one. The earlier two are superseded because the table only holds the current state per key.

9.5 Simple Joins (Combining Two Things)

Stream-Table join — enriching a live feed with reference data

```
// Join a live order stream with a customer KTable to get the customer's name
ordersStream.join(customersTable,
                  (order, customer) -> order + " bought by " + customer.name);
```

Every order looks up the customer's *current* name in the table at the moment the order arrives.

Stream-Stream join

Match two live feeds within a time window — for example, linking a click to a purchase within 5 minutes of each other.

Table-Table join

Like joining two database tables to create a new, combined snapshot — the result is itself a KTable, always reflecting the latest state of both sides.

9.6 Summary Cheat Sheet for Simple Ops

Operation	KStream Behavior	KTable Behavior
`filter()`	Drops or keeps each event.	Drops or keeps the latest state. If an update changes from "bad" to "good", it will now pass through.
`map()`	Changes every single event.	Changes the latest value. Previous versions are forgotten.
`groupByKey()`	Groups all events.	Groups only the latest events. Old updates are overwritten.
`count()`	Running count that increases with every event.	Count updates only when a key's value changes.
`toStream()`	N/A (already a stream).	Converts the table back into a feed so you can see every update as it happens.

A quick code snippet — putting it all together

```
// 1. Read from a topic
KStream<String, String> clicks = builder.stream("clicks-topic");
KTable<String, Long> userTable = builder.table("user-profiles-topic");

// 2. Simple filter (stateless)
KStream<String, String> validClicks = clicks.filter((key, val) -> val != null);

// 3. Simple map to change format (stateless)
KStream<String, Integer> clickLengths = validClicks.mapValues(value -> value.length());

// 4. Group and Count (stateful) - gives a running total per key
KTable<String, Long> clickCounts = clickLengths.groupByKey().count();

// 5. Join stream with table to enrich data (stateful)
KStream<String, String> enriched = validClicks.join(userTable,
    (click, user) -> "User " + user.name + " clicked " + click);
```

FINAL TIP

If your operation does not need a `groupBy` or a `join`, it is **stateless** — fast and easy. If you use `groupBy`, `count`, `aggregate`, or `join`, it is **stateful** — it requires a state store, but gives you much more powerful results.

Chapter 9 Takeaways

- KStream = every event matters and is processed independently; KTable = only the latest value per key matters.
- Stateless ops (filter, map, flatMap) need no memory and run identically on every record.
- Stateful ops (groupBy, count, aggregate, join) need a state store and behave very differently depending on whether they sit on a KStream or a KTable.
- The Golden Rule: a KStream re-runs your logic on every update; a KTable only ever reflects the latest one.
- Joins come in three flavors — Stream-Table (lookup/enrichment), Stream-Stream (windowed), and Table-Table (snapshot combination).

Course Complete — What's Next?

You've now walked through the full arc of building production-quality Kafka Streams applications: from a first "hello world" run, through stateless and stateful DSL operators, exactly-once correctness guarantees, real-time joins against reference data, and automated testing.

Suggested next steps

- Rebuild the WordCount application from scratch, from memory, without referring back to this guide.
- Extend the BankBalance exercise to reject transactions that would make a balance negative, using `transformValues` and a state store.
- Explore windowed aggregations (tumbling, hopping, session windows) — a natural next topic once groupBy/aggregate feel comfortable.
- Read up on Kafka Streams' `StandbyTasks` and interactive queries for building queryable microservices on top of your state stores.

KEEP THIS GUIDE HANDY

Companion Study & Lab Guide — Based on the Course Curriculum

Each chapter's code samples are meant to be copied into your own IntelliJ project and modified — the fastest way to internalize the DSL is to break these examples on purpose and fix them.